This paper describes objective technical results and analysis. Any subjective views or opinions that might be expressed in the paper do not necessarily represent the views of the U.S. Department of Energy or the United States Government.

SAND2020-14055C

DeACT: Architecture-Aware Virtual Memory Support for Fabric Attached Memory Systems

Vamsee Reddy Kommareddy
University of Central Florida
Orlando, Florida, USA
vamseereddy8@knights.ucf.edu

Clayton Hughes Simon David Hammond
Sandia National Laboratories
Albuquerque, New Mexico, USA
{chughes,sdhammo}@sandia.gov

Amro Awad
North Carolina State University
Raleigh, North Carolina, USA
ajawad@ncsu.edu

***Abstract*—[1]The exponential growth of data has driven technology providers to develop new protocols, such as cache coherent interconnects and memory semantic fabrics, to help users and facilities leverage advances in memory technologies to satisfy these growing memory and storage demands. Using these new protocols, fabric-attached memories (FAM) can be directly attached to a system interconnect and be easily integrated with a variety of processing elements (PEs). Moreover, systems that support FAM can be smoothly upgraded and allow multiple PEs to share the FAM memory pools using well-defined protocols. The sharing of FAM between PEs allows efficient data sharing, improves memory utilization, reduces cost by allowing flexible integration of different PEs and memory modules from several vendors, and makes it easier to upgrade the system.**

One promising use-case for FAMs is in High-Performance Compute (HPC) systems, where the underutilization of memory is a major challenge. However, adopting FAMs in HPC systems brings new challenges. In addition to cost, flexibility, and efficiency, one particular problem that requires rethinking is virtual memory support for security and performance. To address these challenges, this paper presents decoupled access control and address translation (DeACT), a novel virtual memory implementation that supports HPC systems equipped with FAM. Compared to the state-of-the-art two-level translation approach, DeACT achieves speedup of up to 4.59x (1.8x on average) without compromising security.

***Index Terms*—disaggregated fabric attached memory systems, virtual memory, security**

I. Introduction

With the ever increasing demand for larger memory capacities, many high-performance computing (HPC) systems nowadays have their nodes equipped with hundreds of gigabytes of memories. For instance, Oak Ridge National Lab's Summit supercomputer has 512GB of DRAM and 96GB of HBM2 per compute node. The driver for increased memory capacity per compute node is the increasing memory needs for current applications and emerging workloads. Most HPC systems typically run many different applications from a variety of domains, each of which will have its own unique resource requirements; some applications may use the whole memory in the node while others may only use a few gigabytes. Nevertheless, most current HPC schedulers allocate resources at the node granularity, and applications with extremely low memory demands can end up reserving nodes with large memories. Unfortunately, the current approach to choose the size of memory per node is based on the maximum footprint (per node) of the applications of interest, which can lead to significant under-utilization of the memory system. Moreover, applications that are not able to fit their memory needs into one node incur additional communication overhead because their computation must be split across nodes. Ideally, compute nodes should have direct access to memories that meet their demands without the need to incur expensive software stack overhead due to message passing libraries.

Recent standards, such as Gen-Z [19] and Compute Express Lanes (CXL) [44], define protocols and interface requirements for accessing memory modules attached to the fast system interconnect. Memory modules that implement memory-semantic protocols and can be readily integrated with the system fabric are typically referred to as Fabric-Attached Memories (FAMs). Protocols defining how to integrate FAMs are being developed through a consortium of major vendors, such as Intel, HPE, AMD, IBM, Lenovo, and VMware [18], [19], [44]. FAMs promise a new HPC architecture where compute nodes can potentially access shared physical memory pools through fast interconnects. In particular, there has been recent industrial interest in architectures where memory modules can be disaggregated from compute nodes, and hence, allows nodes to scale up its memory allocation to the requirements of the workloads run on the node. Such architectures are typically referred to as *memory-centric architectures*. Memory-centric architectures leverage memory semantic protocols to communicate with FAM pools over high-speed interconnects. Owing to memory disaggregation, they promise efficiency, flexibility, and reduced costs. Examples of architectures that resemble memory-centric systems include Facebook's Disaggregated Rack [49], HPE Labs' *The Machine* [16], and Intel's Rack Scale Architecture [37].

Since memory-centric architectures leverage FAMs as physically shared memory pools, multiple compute nodes, potentially running applications from different users, can access pages in the same FAM memory modules. This access model is different from conventional HPC architectures where each compute node has its own memory modules and applications' memory accesses are limited to its own nodes, unless explicitly requested from other nodes through software interface. Therefore, a new

[1]Part of this work was done when Vamsee was working under the supervision of Amro Awad at UCF. Amro Awad is now with the ECE Department at NC State.

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525.

question arises: **who is responsible for access control of FAMs?** Without strict access control mechanisms, malicious operating systems (OSes), applications, and processing elements (e.g., accelerators, SoCs, etc.) can potentially compromise the entire system by accessing the data of other users in the shared FAMs. Note that in this system architecture, there could be compute nodes containing processing elements (PEs) from different vendors. Even if not malicious, these PEs could contain bugs in their internal virtual memory implementation, which can compromise the whole system. Obviously, with such a *wide attack surface*, accesses to shared FAM modules need to be vetted externally, at the system-level, and not rely solely on internal access control within PEs. Pages in shared FAM pools can be managed in two different ways. The first approach is through transparently allocating FAM pages to nodes on-demand, i.e., each compute node has the illusion that it has a contiguous large physical space [38]. Such an approach is similar in spirit to how hypervisors give virtual machines (VMs) the illusion that each VM has a contiguous guest physical memory, which eventually gets translated into the real system physical address through the hypervisor. In this case, a memory broker node is dedicated to set up such translations for each node at the system level. The second approach is to expose each node to the real physical addresses (FAM addresses) and modify the operating system (OS) kernel running on each node to communicate with the external memory broker to allocate FAM pages [38], [43].

Transparent management of FAMs' pages eliminates the need to modify the OS kernel and, most importantly, allows system-level vetting of accesses to FAMs through a second level of memory translation, from the node guest address to the FAM physical address. However, while this is similar to two-level translation in virtualized environments, flexibility, transparency, and security come at the cost of significant performance overheads due to the additional level of translation. In conventional x86 systems, each memory access can require up to four memory accesses for translation, however, when a second level is added, the number of memory accesses can be up to 24 [11]. We observe that significant performance overheads can be incurred when naively implementing state-of-the-art implementations of two-level translation in the context of FAM architectures. Hence, in this paper, we focus on optimizing the implementation of virtual memory support for memory-centric systems.

To minimize the performance overheads of transparent access control and management support for shared FAM pools, we propose *decoupled access control and address translation (DeACT)*. DeACT leverages the architecture layout of memory-centric architectures and the ability to decouple access control from translation. Specifically, DeACT allows *unverified* caching of translations in the small local memories within compute nodes but enforces access control at the system level. By decoupling access control and translation and leveraging part of the local memories in compute nodes as unverified caches, DeACT exploits the high spatial locality of access control metadata (ACM) for each node. By doing this, DeACT brings in significant performance improvements and reduces the number of translation requests significantly, while strictly enforcing access control.

Previous proposals discussed virtual memory support for remote memory architectures [1], [2], [38], [43]. However, these techniques significantly modify the OS [43], propose methods which are non-transparent to the applications and require programmers' intervention [1], limit the use of remote memory merely as a swap space [38] or invalidate virtual memory paging for FAM architectures [2]. None of these approaches consider the performance impact of address translations in FAM. Also, these methods do not consider the security aspects. In contrast, we provide a quantitative analysis of address translation overheads for FAM systems and propose DeACT, a virtual memory support mechanism that leverages the architectural layout of FAM systems to use in-memory caching, which greatly exceeds the capacity of on-chip caches, without modifying OS.

To evaluate our scheme, we use the Structural Simulation Toolkit (SST) [42], a publicly available architectural simulator. We leverage the currently existing disaggregated memory model and memory manager, Opal [34], to model our system. We evaluate DeACT for applications using multiple memory-intensive benchmark suites. DeACT achieves speedups of up to 4.59x (1.8x on average), compared to the state-of-the-art implementation of two-level translation, without compromising security.

In summary, this paper makes the following contributions:

- We discuss and analyze the drawbacks of current memory management approaches in FAM systems; exposing FAM and indirect access to FAM. Exposing FAM to the nodes requires changes to the OS and leads to security vulnerabilities. Introducing indirection in accessing FAM incurs significant performance overheads, up to 20.6x (3.3x on average).
- To effectively manage memory in FAM systems without compromising security, we propose DeACT, a novel hardware support that decouples FAM's access control and address translation.
- We show that, DeACT achieves performance improvement up to 4.59x (1.8x on average) by utilizing local memory to cache decoupled address translations and exploiting high spatial locality of access control metadata.

The organization of the paper is as follows. First, we discuss the threat model, memory management in FAM systems, and motivation in Section II. Section III explains how access control is decoupled from address translation and how unverified caching operates in local memory. Our evaluation methodology and the used applications are discussed in Section IV. The results and sensitivity analysis are discussed in Section V. Section VI discusses the feasibility of huge pages and various other factors in FAM systems. Finally, we discuss the related work in Section VII and conclude our work in Section VIII.

II. Background

In this section, we discuss our assumed threat model, memory management concepts in x86, virtualized systems, and FAM.

A. Threat Model

In our threat model, we assume that compute nodes themselves can have bugs that can be exploited by malicious applications or OSes. Such threats are common and evidenced by the recent vulnerabilities in Intel's processors (e.g., Meltdown [40] and Spectre [33]). Our threat model assumes that a malicious application or OS runs on a specific node that tries to illegitimately access memory pages of other nodes and users in the shared FAMs. Note that we assume a compute node, at any point in time, is owned by a single user (i.e., a user allocated the node to run an application). By exploiting a bug in virtual memory implementation within a compute node or a vulnerability in OS, the attackers can directly map its own virtual space to any physical page in the shared FAM and hence, be able to access it freely. Therefore, to minimize the attack surface, an additional level of access control needs to vet accesses that come from compute nodes to ensure that they are for pages belonging to the node. Thus, any pages in FAM that are considered exclusive to a node, must be protected from any access by other nodes as long as such pages are allocated. Attacks such as timing side-channel, covert-channel and physical attacks are beyond the scope of this paper and can be addressed with many available solutions based on the system nature. Similar to most HPC systems, we disallow co-locating resource allocations on the same node, which minimizes the risk of information leakage (within a node). In summary, we mainly focus on enforcing access control on shared FAMs, to limit the impact of vulnerabilities within compute nodes on other compute nodes' data. Such protection is analogous to security guarantees provided by virtual memory for applications running on a native system, but at the node level. Our threat model trusts the memory and fabric, i.e., memory provides nodes with the requested data and does not try to give them data from other locations. Similarly, the fabric will not change the address in a request after it has been vetted by access control.

B. Hierarchical Page Tables

Hierarchical (multi-tier) page tables are commonly used in modern servers due to their performance, dynamic growth, and scalability. In such settings, a virtual address is provided as an input to the translation process, then the offsets (derived from virtual address) are used to index each level to obtain the address of the next level. Finally, the last level, typically called Page Table Entry (PTE) level, has the actual translation entry, i.e., the corresponding physical address and the access permissions. However, to reduce translation overheads, hardware-support for memory management, typically implemented as the Memory Management Unit (MMU) and maintained by the OS, is provided. The MMU is responsible for caching the translations, i.e., PTEs, in TLBs. Moreover, MMU can also cache the contents of different levels of the page table in what is called page table walking (PTW) caches [11]. Finally, MMU is responsible for walking the page table in case of TLB miss to complete the translation process.

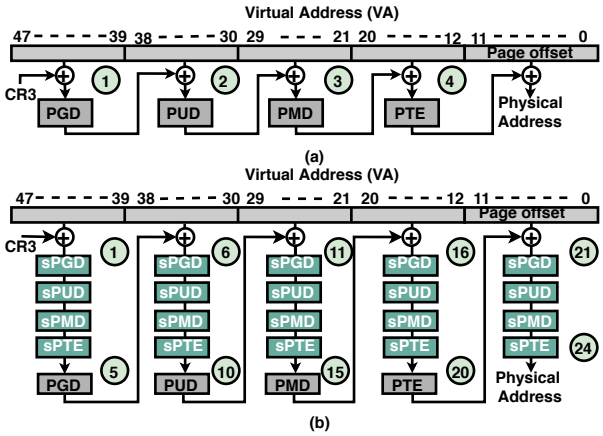


Fig. 1: Page table walking in (a) x86 and (b) virtualized systems.

As shown in Figure 1(a), for every TLB miss, the page table is walked. In x86-64 systems, a 4-level page is typically used, and the levels are called PGD, PUD, PMD and, PTE, respectively. Therefore, each memory access needs additional four memory requests to walk the page table. The root of the page table of the process currently executing on the core is loaded in Control Register 3 (CR3) in the core. In each virtual address, the bits beyond the page offset (typically 12 bits) are divided into multiple sections (9 bits each) where each section is used to index a specific level in the page table. Finally, the last-level page (PTE) has the actual page mapping. Hence, for every TLB miss, the entire page table is walked which will incur an additional four memory accesses.

In virtualized systems, hypervisors like Xen [50] are responsible for maintaining multiple guest systems and managing their memory. For such systems, nested paging is one approach in which two page tables are maintained: one to convert virtual address to guest address and the other is a nested page table to convert guest address to system physical address. Each level of the guest page table has to walk the nested page table which requires four more memory accesses per guest page table level, Figure 1(b). Hence, 24 memory accesses are required to fetch the translation. This leads to huge overhead and hence, Bhargava et al. [11] proposed PTW caches, nested TLBs, and nested PTW caches to reduce the number of memory accesses to translate an address. PTW cache unit caches the intermediate level address translations and helps in reducing the average number of PTW steps.

C. Fabric Attached Memory (FAM)

Recently, there has been a push towards disaggregating resources (such as memory modules) [1], [2], [35], [38], [39], [43] and connecting them through system fabric [14], [18], [20], [30]. Fabric-attached memories (FAMs) are memory

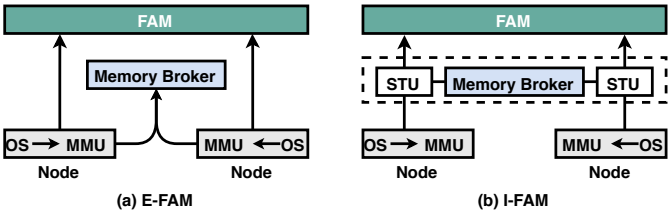


Fig. 2: Two ways of managing memory in FAM systems.

modules that are attached to system fabric which can be accessed by multiple nodes through conventional load/store operations. FAMs allow disaggregating memory pools and make them accessible to compute nodes through fabric using well-defined memory access protocols, such as Gen-Z [19], CXL [44]. Managing disaggregated FAM uniquely combines the challenges of managing shared FAM and multi-level memory (local memory and remote memory). On one hand, nodes should be prohibited from illegitimately accessing data belonging to other nodes. On the other hand, OSes running on nodes are agnostic to the actual status of the FAM and thus, would instead manage a hypothetical large node physical memory which translates into FAM. However, to enforce access control and permissions, translating nodes' physical address into FAM address should occur at the system-level (off-the node), not at the node-level. By doing so, any malicious OS or node will be limited to accessing the data allocated to the node instead of the whole global memory.

Adding a translation layer at the system-level allows running unmodified OSes on nodes and enforcing access control, but incurs significant performance overheads. Lim et al. [38] explored a two-stage address translation system for disaggregated memories. For the rest of the paper, we call such a scheme *Indirect FAM (I-FAM)*, since FAM is accessed indirectly. In I-FAM, a simple system translation unit (STU), similar in spirit to *Gen-Z memory management unit (ZMMU)* [21], can be implemented in a router connected directly to the node, in the memory blade or as an independent network element, as shown in Figure 2(b). STU is responsible for caching system-level translations, i.e., node address to FAM address, and access permissions. Moreover, STU is capable of sending address translation service requests (similar to PCIe's [23]) or request physical pages from the system-level memory broker (in case of unmapped addresses). Note that STU is similar in spirit to the ZMMU [21]. Exposing the FAM directly to nodes' OSes would significantly improve the performance by removing the additional translation layer, Figure 2(a). We refer to such a scheme as *Exposed FAM (E-FAM)*. In E-FAM, OSes need to be patched to communicate with the global memory manager node (e.g., through MPI interface) to coordinate memory management with other nodes. While E-FAM provides translation overheads as low as native systems, it requires modifying OS [43] and enormously enlarges the attack surface; any malicious node/OS can map its address space into any location in global memory, hence leaks data from other nodes.

D. Motivation

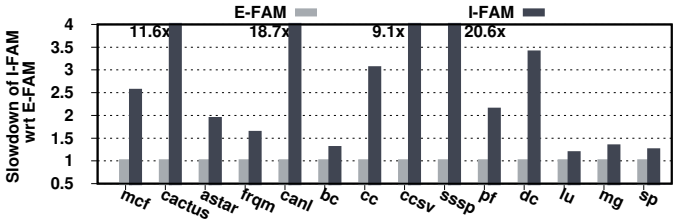


Fig. 3: Normalized performance with respect to E-FAM.

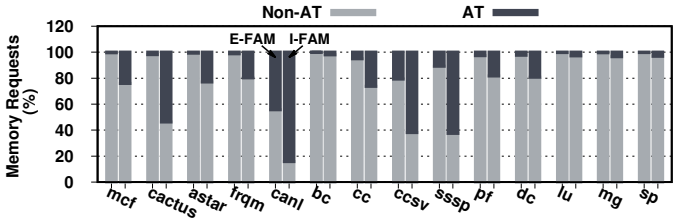


Fig. 4: Breakdown of percentage of address translation (AT) and non address translation (Non-AT) requests observed at FAM in I-FAM and E-FAM systems.

Figure 3 shows slowdown in I-FAM compared to insecure E-FAM wherein no indirection is needed. We observe a performance degradation of 20.6x for *sssp* benchmark (details about the methodology and the benchmarks are discussed in Section IV). The slowdown is attributed to the increased address translation requests observed at FAM due to indirection at the system-level, Figure 4. For instance, the percentage of address translation requests for *canl* benchmark is 44.36% in E-FAM, however, this increases to 84.13% in I-FAM. Also, we note that benchmarks that are not sensitive to address translations become highly sensitive to address translations in I-FAM. Address translation requests increase from 1.81% to 53.69% for *cactus* benchmark. Clearly, I-FAM brings in significant performance overheads. The goal of DeACT is to design a FAM architecture with better performance, without sacrificing security and with minimum or no changes to the OS. A comparison of baseline FAMs and our proposed DeACT FAM approach is shown in Table I.

TABLE I: FAM Architectures Comparison.

Architecture	Performance	Avoid OS Changes	Security
E-FAM	✓	✗	✗
I-FAM	✗	✓	✓
DeACT	✓	✓	✓

III. Design

In this section, we describe our proposed approach DeACT to provide secured and efficient memory management support for FAM systems.

A. DeACT Overview

When designing support for virtual memory, we aim at abstracting away the details of the global memory from nodes' OSes, however, while enforcing isolation and minimizing translation overheads. To do so, we adopt a two-layer approach where each node's OS manages an imaginary flat node physical memory. The node physical memory range can be thought of as a range of two different NUMA zones, one zone (low addresses) corresponds to the local DRAM and the other zone (high addresses) corresponds to the FAM. With such a design, each node's OS manages its memory allocations oblivious to the actual status of FAM. While such an approach abstracts away the complexity of managing a shared resource (memory), it adds significant performance overheads due to two levels of indirection. Therefore, we need novel mechanisms to improve the performance of such a design without compromising security.

One major observation we make is that access control can be decoupled from the translation process. In particular, the translation from node address to FAM address can be sped up significantly by caching the translations at node-level memory. Later, if the translation of a specific node address exists locally in the node, the global memory request is forwarded to the FAM, with the obtained/cached FAM addresses. In other words, the node can provide the final FAM addresses it needs to access. As the reader can expect, the access control is offloaded to the off-the node components, e.g., STU units. Since the access permissions need to be checked for the specific FAM address provided by the node, we dedicate specific parts in the FAM to store ACM of FAM. Such parts are known for STUs and the addresses of the ACM of any FAM page can be derived merely from the FAM address. For instance, assume we want to keep a 16-bit ACM for each 4KB page and assume the metadata starts at address $MTAdd$ in FAM. To read ACM of FAM address *X*, we read the 64-byte block at address $MTAdd + \frac{X}{4096 \times 32}$ (division by 32 is needed to get the base address of 64-byte block for 16-bit ACM). For simplicity, the metadata of each 4KB page is the node ID of the node that owns that page and read/write permissions. Read and write fields consume two bits and the rest of the bits (14) are allocated for the node ID. Nodes can choose to execute pages read from the FAM. However, while executing instructions, the access permissions of the executable pages are verified by TLBs, within the node. FAM pages could also get shared between the nodes. Thus, we use all the node ID bits of the page metadata set to 1 to indicate a shared page. Hence, we can have up to 16383 nodes supported in the system.

Since pages can be shared by a subset of nodes, just indicating a page is shared is insufficient. Therefore, we use a bitmap-like scheme to indicate which nodes are allowed to access a specific page. However, since having a bitmap for each 4KB can introduce significant overheads, we limit shared pages to 1GB physical pages. For each 1GB physical page in global memory, we have a corresponding 64K bits bitmap (8KB) in the metadata region. Since such overhead is negligible

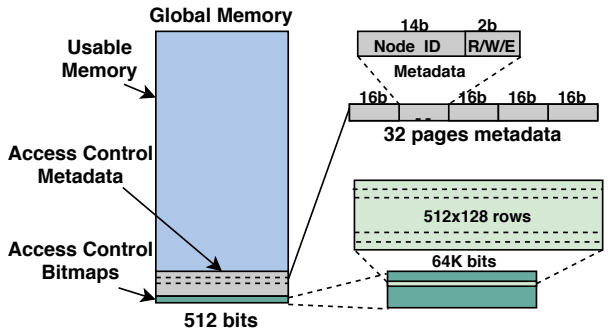


Fig. 5: Page access control metadata and bitmaps in FAM.

(less than 0.0001%), and to enable easier indexing of metadata, we dedicate a bitmap for each 1GB physical region regardless of being used as a shared page or not. Therefore, when ACM is accessed, if the node ID bits of the metadata indicates a shared page, we immediately fetch the corresponding parts of the bitmap to check if the node has access permissions. In contrast, if the node ID bits do not indicate a shared page, we simply compare its value (owner node ID) with the ID of the requesting node, to verify the legitimacy of the access. Note that when a shared page is allocated (or becomes shared), all of its node ID bits in the metadata fields correspond to its 4KB chunks (sub-pages) are set to shared, i.e., `0xfffd`. When the page is shared, the last two bits of the metadata field indicate read and write permissions assigned to the node. This enables enforcing mixed access permissions for nodes sharing a page. For instance, a specific subset of nodes is allowed to read and write to the shared page and the rest of them can only read the shared page.

One obvious optimization to reduce the overheads of obtaining such ACM is to cache them. However, since such metadata must be enforced by FAM managers, not the node, such metadata should be only cached outside the nodes and inaccessible by the nodes or their own OSes. Therefore, we opt for caching such metadata in STUs. Such STUs can potentially have a small lookup table, similar to TLBs. As mentioned earlier, such STUs can be added to the global memory blade or simply at the first router/switch that connects a node to the system fabric. It is also important to note that such metadata has very high spatial locality, a single 64B block covers 32 4KB pages, i.e., 128KB region for a 16-bit ACM. Therefore, even a very small TLB-like cache can save a significant number of reads to access control metadata. While beyond the scope of this paper, in encrypted memories, if each node uses a unique memory encryption key, we could allow read requests without checking access control; writes can tamper with data but reads are useless if the node has a different unique key, and thus, no need for enforcing access control for reads.

As we now understand how our decoupled access control works, we will discuss how we accelerate the translation process. To speed up the translation process, we (a) propose node-level unverified caching of system-level translations: We

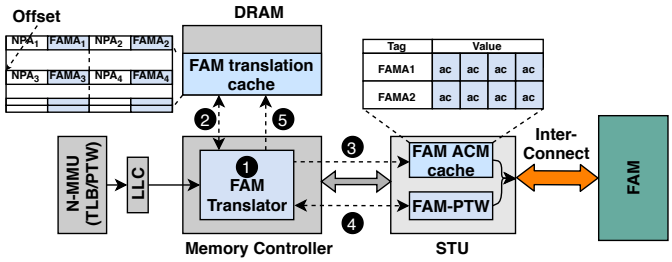


Fig. 6: DeACT FAM schematic.

notice that a very small portion of local memory can be used to cache system-level translations, which will be later sent for verification at system-level, (b) efficiently cache ACM in STU.

B. System Overview

Figure 6 shows a schematic overview of our proposed design. Decoupling the metadata from page mapping enables the system-level translations to be cached in the local memory. Hence, we maintain a FAM translation cache in the DRAM. We add a *FAM translator* (1) in the memory controller to map node addresses to FAM addresses by accessing the FAM translation cache (2). Although node addresses are mapped to FAM addresses by the FAM translator unit it is still a partial translation since accesses have to be verified. To complete the mapping, the FAM accesses are verified by the STU (3). Hence, unlike I-FAM, DeACT requires two steps to translate a node address and verify the access.

For a translation miss, the FAM page table has to be walked. In our design we let the walking to be done by the STU since we observe that the overhead of including FAM PTW inside the node is costlier than the benefits. Firstly, due to security reasons, we aim at a clean separation of address translations and ACM, if the fabric translations are cached inside the node. Hence, if the intermediate translations are also cached within the node, the ACM for intermediate page tables should also be decoupled. However, the two-step process required to complete the mapping, delays address translations significantly, considering four memory accesses during PTW. That is, at every intermediate level, the ACM should also be fetched from the memory incurring additional memory accesses. Secondly, since FAM translation cache size in local memory is significantly higher than the STU cache size, we observe a hit rate of more than 90% in FAM translation cache in the local memory. Hence, walking the FAM page table within the node would unnecessarily increase the complexity without many benefits. Also, it would increase the complexity of the memory controller. Thus, we apply DeACT only to the last level of the page table (PTE). Therefore, during a FAM translation miss, the FAM translator forwards the request to the STU, which walks the FAM page table and fetches the entry on behalf of the FAM translator (4). After receiving the missed translation, the FAM translator maps the pending requests and then updates the FAM translation cache in the local memory(5).

C. FAM Translator

The idea of the FAM translator in the local memory controller is to translate node addresses to FAM addresses without verifying memory accesses. Functionalities of FAM translator are: (a) fetching the translation from the FAM translation cache (b) matching the tag (c) handling translation hits and misses (d) handling off-the node responses and (e) updating FAM translation cache in the local memory.

Accessing DRAM for Translation: To fetch the translation from the local memory, (a) in Figure 7, the FAM translator calculates the local memory address by adding starting address of FAM translation cache to the offset, Figure 6. Offset is dependent on the type of the FAM translation cache in the local memory. For simplicity, we use a four-way associative cache. This is because memory access granularity is 64-bytes and each mapping entry requires 104 bits; 52 bits for tag (node page) and 52 bits for value (FAM page), for a page size of 4KB. Single memory access fetches four entries. Thus, offset is obtained by performing a modulus operation on node page number with the number of FAM translation cache sets.

Tag Matching: After fetching the translations from the local memory, FAM translator matches address tags using comparators, (b) in Figure 7. We add four comparators and a multiplexer to perform tag matching concurrently. If none of the tags match, the output of the multiplexer is set to 0. This takes just one cycle to match the tag but the number of comparators required is 4x more compared to using just one comparator when four tags are matched serially in four cycles. However, these additional comparators add up minutely to the overall hardware cost and area.

Handling Translation Hits: When any of the tags, fetched from the FAM translation cache, match with the required node page address the multiplexer outputs the respective FAM address. FAM translator replaces the node address with the FAM address and forwards the request. However, before forwarding the request to STU, the FAM translator identifies if the request is expecting any response back from the FAM. If so the FAM address to node address mapping is stored in the *outstanding mapping list*, (c) in Figure 7. This is because FAM responses contain data tagged with FAM addresses and nodes only deal with node addresses. *Outstanding mapping list* is used to convert the FAM address to node address during FAM response. Since the number of outstanding requests is limited (128 requests in our evaluation setup, Section IV) the number of entries in *outstanding mapping list* is also limited. In I-FAM this list is maintained in STU. But since the FAM translations are performed within the node and STU does not understand node addresses in DeACT, this list is maintained within the node.

Handling Translation Misses: A translation is identified as a miss if the output of the multiplexer is zero, (d) in Figure 7. During a miss, FAM translator forwards the request to the STU to walk the page table. FAM PTW unit of STU retrieves the node address from the missed request and walks the page table. After the page table is walked, the STU translates the

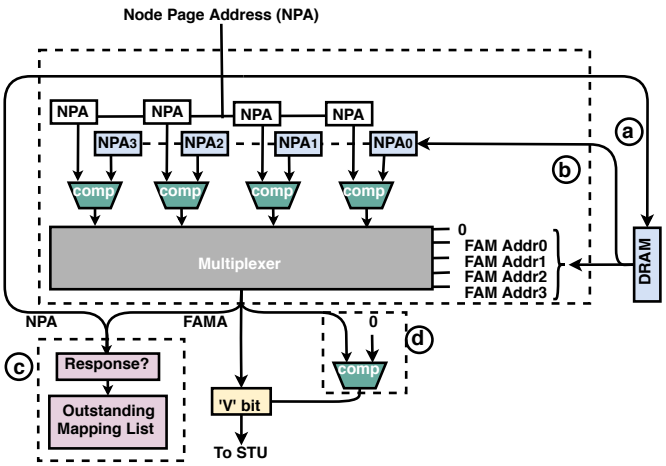


Fig. 7: FAM translator.

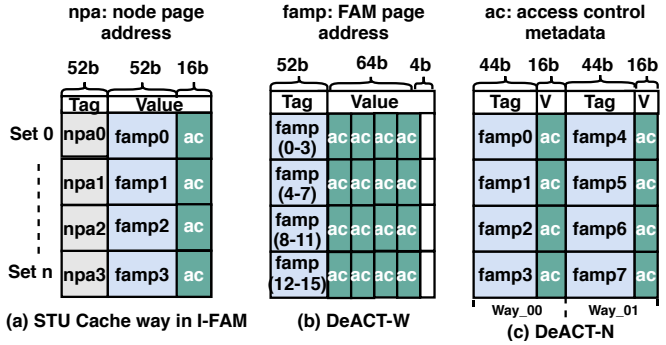


Fig. 8: ACM organization in STU cache way in (a) I-FAM and (b,c) DeACT.

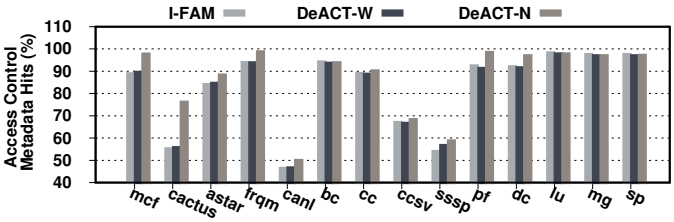


Fig. 9: Access control metadata hit rate.

node address to the FAM address and then verifies the access to forward the missed request to the FAM. Also, the STU sends the page mapping to the FAM translator for updating the FAM translation cache and to register the mapping in the *outstanding mapping list* if needed.

STU receives two types of requests from a node, mapped and not mapped requests. Mapped requests are those whose node address is translated to FAM address by FAM translator. For such requests, STU verifies FAM access permissions. On the other hand, STU walks the page table for not mapped requests using the node address from the request address field. To make STU distinguish between the two types of requests we add a verification ('V') flag to the request packet. This flag is set by the FAM translator unit if the mapping is successful and is reset for a missed translation. Using the 'V' flag STU either forwards the request to the verification unit or to the PTW unit.

Handling off-the node memory responses: FAM translator segregates responses into two types (a) memory response and (b) mapping response. Memory responses are forwarded to the last level cache by fetching the node address from the *outstanding mapping list*. Mapping responses are received from the STU PTW unit. FAM translator updates the FAM translation cache during mapping response.

Updating FAM Translation Cache: Since the granularity of memory access is 64 bytes each access to FAM translator operates on four FAM mappings. To update the FAM translation cache, the FAM translator has to write to one of the four mappings fetched. Hence, during a translation cache update, the FAM translator reads 64 bytes from the local memory, updates one of the entries, and writes back 64 bytes. For simplicity, we randomly selected one of the four entries to replace. It is possible to implement different cache replacement policies but such policies require additional DRAM space; to store mapping status, and additional writes to the DRAM; to update mapping status for every FAM access.

D. FAM Access Verification

Memory accesses verification is performed by the STU in DeACT. STU verifies the memory access by a) checking if the page being accessed is assigned to the node, using node ID in the metadata and b) checking the read and write permissions from the metadata. However, the metadata is not provided by the node since we maintain the page metadata in the memory off-the node to provide security, Section III-A. Hence, for STU to verify the FAM access it needs to fetch the page metadata from the memory, Figure 5.

Since STU is off-the node, it can be used to cache ACM. STU in I-FAM caches both FAM page mapping and ACM together, Figure 8(a) (52 bits for the tag (node page address) and 52 bits for FAM page address and 16 bits for ACM). However, STU in DeACT caches only ACM. Although DeACT reduces the frequency at which the page table is walked by leveraging local memory to store FAM mappings, it introduces additional memory access for the ACM. Hence, to reduce the number of accesses to FAM for ACM, we explore organizing ACM in the available space, after decoupling the page mapping from the STU cache in DeACT.

Way-level contiguous organization (DeACT-W): This is a simple organization wherein the space available in each cache way, after removing the address mapping, is used to cache ACM of contiguous pages, Figure 8(b). Since ACM is 16 bits and the space available is 52 bits (FAM page address), four contiguous pages ACM is stored in one cache way. For instance, ACM for pages from 0 to 3 are stored in one cache way and 4 to 7 are stored in a different cache way. Hence, caching of ACM increases by four times.

Non-contiguous organization (DeACT-N): With DeACT-W

we observe ACM hit rate of almost 90% for most of the benchmarks, Figure 9. However, ACM hit rate for benchmarks like *canl*, *sssp* and *cactus*, which are sensitive to address translations, is less than 60%. This is because STU in DeACT-W achieves a higher hit rate when spatial locality, while accessing memory, is higher. However, since FAM is shared by multiple nodes, memory allocation is random and hence, has poor spatial locality while accessing memory. Thus, instead of organizing STU ACM cache to cache contiguous pages ACM, we organize STU ACM cache to cache ACM of non-contiguous pages i.e., the free space is used to store tag and ACM pair of another page which is either contiguous or non-contiguous, Figure 8(c).

Since each tag and ACM pair needs 68 bits; 52 bits for the tag and 16 bits for the ACM, the available free space, 52 bits, is not sufficient to store an additional pair. Thus, to fit ACM for two different pages within the same way of a set (within the available space) we confine the number of tag bit to 44. With 44 tag bits STU can cover up to 8 tera pages metadata[2] and hence, each node can access 32 petabytes of memory unlike 16384 petabytes with DeACT-W. However, 32 petabytes is also significantly higher for a node. Thus, each cache way is subdivided into two sub-ways (way_00 and way_01 for way 0) and each sub-way has a tag and ACM. This increases the total number of ways for a set and matching the tags of sub-ways is similar to matching the tags of different ways in a cache. Organizing ACM in STU cache in this manner doubles the caching of ACM and unlike DeACT-W, DeACT-N stores non-contiguous page ACM.

The non-contiguous organization of ACM in the STU cache increases the hit rate from 90% to almost 99% for most of the applications. Also, the hit rate for address translation sensitive benchmark, for instance, *cactus*, increases from less than 55% to almost 76%. The improvement compared to DeACT-W is due to random accesses to FAM (see Section V).

IV. Methodology

To evaluate our design we used a decoupled memory model implemented in Structural Simulation Toolkit (SST) [34], [42]. SST is an event-based cycle-level simulator that has been proven to be one of the most reliable simulators for large-scale systems due to the scalability and modular design of its components. SST includes multiple simulation modules for various components. To evaluate FAM architectures a FAM manager (memory broker), Opal [34], was developed in SST. We modified SST memory management unit, Samba [8] and Opal [34] modules to model our design. We modeled an STU component in SST to translate node addresses to fabric addresses and to verify FAM accesses. As our approach focuses on accelerating the address translations, we validate our approach by calculating the performance of the system in terms of instructions per cycle.

Table II shows system simulation parameters. We simulated 4 cores and each core can serve up to two instructions per cycle with a frequency of 2GHz. Each core is configured to execute a minimum of 100 million instructions of an application execution during its HPC-relevant kernels. L1, L2, and L3, caches are inclusive with sizes 32KB, 256KB, and 1MB respectively. Local memory, is 1GB DRAM [43]. For FAM, as projected by many system vendors (e.g., as in The Machine of HP Labs [16]), we use emerging non-volatile memory (NVM) due to its high-density, ultra-low idle power mixture of storage and memory characteristics [5]–[7], [15], [36]. Our assumed FAM is 16GB NVM[3]. We simulated a fabric network to connect to NVM memory with a network latency of 500ns, modeled after recent research and public projections for fabric interconnects [26], [43], [46]. Two levels of TLBs, each of which is simulated with 32 and 256 entries within the node. Since STU is an external hardware per node, we have restrictions over adding additional hardware. Hence, to avoid significant hardware overhead, we implemented STU to cache 1024 page table entries with 128 sets and associativity of 8, similar to Haswell Xeon L2 TLB design [28]. However, we also evaluated DeACT by varying STU cache size. For optimization proposed by Bhargava et al. [11] we used 32 PTW cache entries. The proposed FAM translation cache size in DRAM is 1MB.

TABLE II: System Configuration

Node	
CPU	4 Out-of-Order cores, 2GHz, 2 issues/cycles, 32 max. outstanding requests
TLB	2 levels, L1 size: 32 entries, L2 size: 256 entries
L1	Private, 64B blocks, 32KB, LRU
L2	Private, 64B blocks, 256KB, LRU
L3	Shared, 64B blocks, 1MB, LRU
Local memory	DRAM, Size: 1GB
STU	
Cache	Size: 1024 entries, associativity: 8
Fabric Network	
Latency	500ns
Fabric Attached Memory (NVM)	
Capacity	16GB
Latnecy	Read 60ns, Write 150ns
Banks	32
Outstanding requests	128

Since our focus is on HPC applications we evaluated benchmarks from different benchmark suits, as shown in Table III. Our selection of benchmarks from these benchmark suits are based on (a) the benchmark should have a minimum of 5 misses per kilo instructions (MPKI) (b) should be compatible with the simulation setup (c) the performance degradation with I-FAM should be more than 15% compared to E-FAM since we observe application which does not get impacted much by introducing indirection degrades it performance with DeACT, explained in Section V. Considering these criteria we have evaluated 29 benchmarks and zeroed in on 14 benchmarks that are meeting the requirements. Selected benchmarks with

[2]Note that these numbers are based on the tag and data bits assumed in the STU cache of I-FAM as shown in Figure 8(a)

[3]In reality local memory is in GBs and global memory is in TBs or PBs. However, given slow simulation speeds, we scale down the memory sizes and among the total application's memory (average of 309MB during the simulation period), 20% is allocated from the local memory and 80% is allocated from the FAM.

TABLE III: Applications

Benchmark Suite	Application	MPKI
SPEC CPU 2006 [31]	Mcf	73
	Cactus	60
	Astar	9
PARSEC [12], [13]	Freqmine (frqm)	16
	Canneal (canl)	57
Intel GAP [10]	Betweenness Centrality (bc)	113
	Connected Components (cc, ccsv)	56, 130
	Single-Source Shortest Paths (sssp)	144
Mantevo [32]	Path Finder (pf)	41
NAS [9]	DC	49
	MG	99
	SP	141

their respective MPKI are shown in Table III. We used short forms to represent applications. The short forms are next to the application names in Table III. Connected components graph analytic benchmark has 2 variants *cc*; which uses Afforest subgraph sampling algorithm [47], and *ccsv*; which uses Shiloach-Vishkin algorithm [45].

V. Results

The goal of DeACT is to provide security from other tenants in FAM systems without significantly impacting the performance. Hence, we compare DeACT with two baselines, E-FAM and I-FAM. E-FAM is not secure but has better performance. I-FAM is secure but performs poorly (remember that I-FAM is similar to the optimization proposed by [11]).

A. FAM Address Translation Hit Rate

Figure 10 depicts address translation hit rate while accessing FAM in I-FAM and DeACT. The hit rate corresponds to the number of mapping entries that are cached in I-FAM and DeACT. DeACT has a significantly higher hit rate (more than 90%) because the FAM translation cache in local memory can cache a significantly higher number of mapping entries than limited entries that can be cached in I-FAM using STU cache. For instance, the hit rate for *canl* benchmark is as low as 46.44% in I-FAM. However, with DeACT the hit rate is improved to almost 95.88%. Hence, only 4.12% of the FAM accesses require page table to be walked.

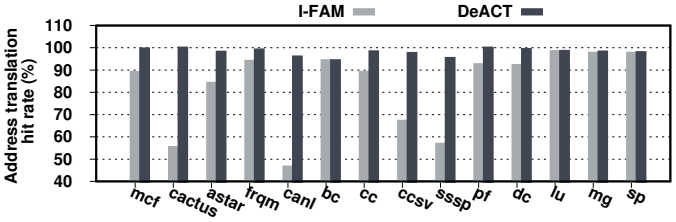


Fig. 10: Address translation hit rate in I-FAM and DeACT

B. Address Translation Requests at FAM

Although the frequency at which PTW is reduced with DeACT, it introduces additional memory access for ACM. ACM is cached at STU cache and address translation is cached in the local memory. Hence, in DeACT address translation and ACM has different hit rates. As shown in Figure 9, ACM hit rate in DeACT-W is not improved compared to I-FAM due to poor spatial locality. Hence, the reduced number of address translation requests observed at FAM in DeACT-W, Figure 11, is only due to reduced frequency of walking the page table and it also includes additional memory access for ACM. However, ACM hit rate is improved in DeACT-N due to the non-contiguous caching of ACM. We observe address translation requests sent by the node to the FAM are reduced from 23.97% to 11.82% with DeACT-W, and this further reduces to 1.77% with DeACT-N.

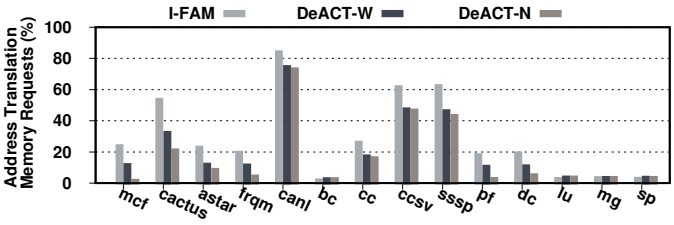


Fig. 11: Percentage of address translation requests at FAM.

C. Impact of DeACT on Performance

In this section, we show how DeACT performs compared to E-FAM and I-FAM. E-FAM performs better than I-FAM and DeACT, hence, we show our results with respect to E-FAM in Figure 12. As mentioned in Section II-D, I-FAM slows down the system performance significantly. Our experiments demonstrate that DeACT can potentially bridge the gap between E-FAM and I-FAM. For instance, *mcf* slows down by 0.39x in I-FAM compared to E-FAM. DeACT-W performance is 0.7x wrt E-FAM, improving the performance by 1.79x compared to I-FAM. Further with DeACT-N the performance is improved by 2.55x and is just 0.92x times slower than E-FAM. This improvement is attributed to the increased FAM address translation hits, using local DRAM and increased ACM hits in STU, leading to decreased accesses to FAM for page table requests by the node, as shown in Figure 11. The inequality between performance improvement and reduction in the observed percentage of address translation requests at FAM is because the local memory is accessed for every FAM access for the translation. In Figure 12, DeACT-A indicates the performance improvement achieved by only caching address translations in the local memory and not modifying STU to store ACM of multiple pages per entry as with DeACT-W and DeACT-N. We observe the performance improvement of DeACT-A and DeACT-W is almost the same since FAM pages are allocated randomly. DeACT-W performs better when the spatial locality is higher while accessing memory. However, with random FAM allocation, the benchmarks have poor spatial locality while accessing memory. The additional improvement achieved by DeACT-N compared to DeACT-A is due to caching ACM of two non-contiguous pages within an entry in the STU.

For *canl, ccsv and sssp* benchmarks we observe a significant percentage of FAM address translation misses in I-FAM, and hence, we observe an increase in the percentage of

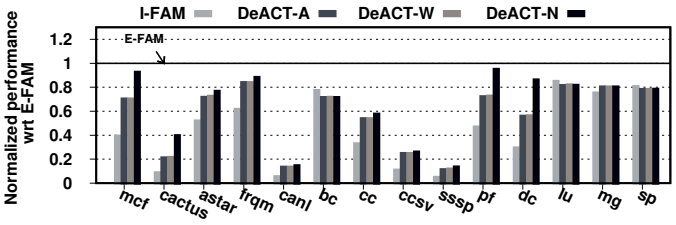


Fig. 12: Normalized performance with respect to E-FAM.

address translation requests to FAM. The performance for such benchmarks even with DeACT-N is slower compared to E-FAM, 0.14x for *canl*. However, compared to I-FAM, DeACT-N achieves a speedup by 2.7x for these benchmarks.

DeACT-N achieves a maximum performance improvement of 4.6x for *cactus* benchmark. However, DeACT either does not improve or degrades the performance for *bc*, *lu*, *mg* and *sp* benchmarks. Because these benchmarks are very less sensitive to indirection in I-FAM, Figure 11, as they have better address translation hit rate, Figure 10. However, in DeACT the DRAM has to be accessed for address translations, which is costlier than accessing the STU cache. Also, the benchmarks have to go through two serial steps for address translation and access verification, unlike a single step for the same in I-FAM. Hence, DeACT is better suitable for benchmarks which have a significant impact on performance with I-FAM. In total, we observe an average performance drop of 69.7% with I-FAM and with our proposed mechanism the performance degradation is 35.3% compared to E-FAM. Hence, DeACT improves I-FAM by 80%.

D. Sensitivity Analysis

The impact of performance in FAM systems is dependent on various factors. In this section, we show how DeACT behaves under various system configurations. The default system parameters are as shown in Table II. Note that for sensitivity results we show the geometric mean of the evaluated SPEC CPU 2006, PARSEC, and GAP benchmarks separately. Also, among NPB benchmarks, we observed *dc* is the only benchmark that has a significant performance impact in I-FAM even under various circumstances. Hence, going forward we show sensitivity results only for *dc* benchmark among NPB benchmarks. Also, since DeACT-N improves the performance more than DeACT-W, we focus on DeACT-N scheme.

1) STU Cache Size and Associativity: One of the main factors which impact the performance of I-FAM is the size of the STU cache. STU is hardware maintained outside the node to enforce system access control and page mapping. The number of entries STU can cache is limited, since we are proposing STU per node and is implemented in the routers connected to the nodes. Adding more entries indicates adding more hardware which increases the hardware budget and complicates routers. In our experiments, STU caches 1024 entries. However, we study DeACT by varying STU cache size from 256 entries to 4096 entries. Figure 13 shows performance speedup compared to I-FAM by varying STU cache size. As STU cache size decrease the speedup with DeACT is significantly high, 4.68x with 256 entries for *dc* benchmark. However, as the cache size increase, the performance improvement is confined which is obvious. The speedup reduces from 3.45x to 1.75x when STU cache size is varied from 256 entries to 4096 entries for PARSEC benchmarks. Higher STU cache size has a higher hit rate and hence, less address translation requests to FAM. However, higher cache size leads to more hardware overhead.

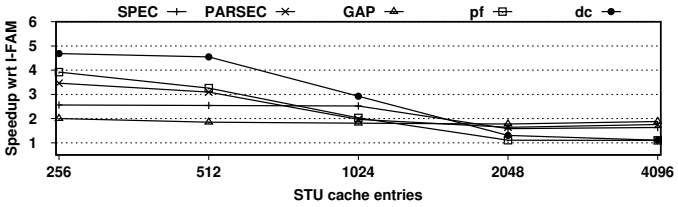


Fig. 13: Performance improvement wrt STU cache size.

Although we do not show here, we also evaluated DeACT by varying STU cache associativity. We observed that as the associativity increases, the performance improvement with DeACT decreases and gets saturated. When associativity is four the performance improvement is 3.26x for *dc* benchmark and is 2.66x when associativity is 32. The speedup is 2.5x when associativity is greater than 32 for the same benchmark. Similarly for PARSEC benchmarks the speedup is 2.18x, 1.83x, and 1.81x when associativity is 4, 32, and greater than 32.

2) Access Control Metadata Size: ACM size is a key design aspect of DeACT as the number of nodes supported by FAM systems is dependent on metadata size, refer to Section III-A. With 16-bit metadata size FAM systems can host a total of 16383 nodes and with 8-bit metadata 8191 nodes are supported. When ACM is 8 bits STU in DeACT-W can cache metadata of eight consecutive pages, with 16-bit metadata STU can cache metadata of four consecutive pages, and with 32-bit metadata STU can cache metadata of two consecutive pages, increasing the amount of metadata cached by 8x, 4x and 2x respectively compared to I-FAM. However, we observe that the performance improvement is almost the same for these three scenarios, Figure 14. This is because, as asserted, although caching of ACM increases in DeACT-W, it caches only ACM of contiguous pages and since the allocation of FAM is random excess caching of ACM is not leveraged.

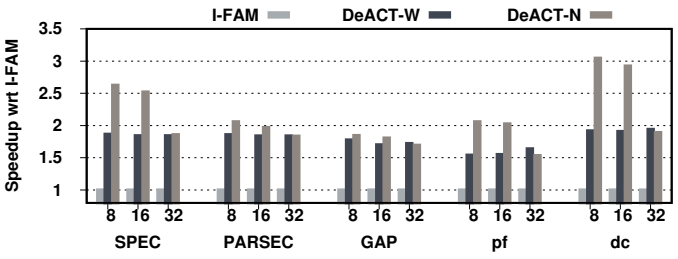


Fig. 14: Metadata size effect on performance.

When ACM size is 8 bits, tag and metadata pair in DeACT-N requires 52 bits (44 bits for tag and 8 bits for ACM, see

Section III-D). Hence, in a single way, STU cache can store two tag and ACM pairs, similar to when ACM is 16 bits. However, the amount of memory required to store ACM of all the pages is reduced to half. As an experimental model, we further reduce the size of the tag to allocate three pairs of tag and ACM per STU cache way, when ACM is 8 bits. When ACM is 32 bits STU can cache only one tag and ACM pair in DeACT-N. We observe that as caching of tag and ACM pairs in the STU cache way increase, from one to three, the performance improvement with DeACT-N also improves. For instance. the system performance improves by 2.62x, 2.52x, and 1.85x when three, two, and one pairs of tag and ACM are cached in each STU cache way, for SPEC CPU 2006 benchmark. It is interesting to note that when only one pair of tag and ACM is stored in the STU cache way the performance improvement is less than or equal to DeACT-W. This is because when only one pair of tag and ACM is cached in each STU cache way the performance improvement is only due to increased address translation hits in FAM translation cache and ACM hit rate is same as I-FAM in DeACT-N.

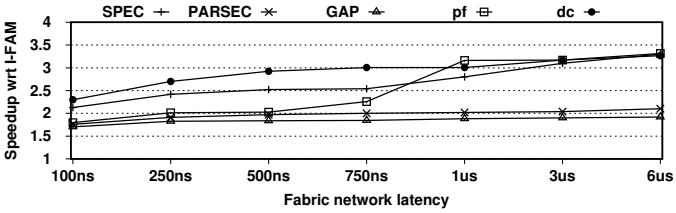


Fig. 15: The impact of fabric latency on performance.

3) Fabric Latency: We propose DeACT for FAM architectures specifically and hence, one of the crucial parameter to consider for such architectures is fabric network latency. In our approach we considered 500ns as fabric latency. However, fabric networks are being explored intensively by various fabric providers [18], [20], [22], [44]. Previous approaches considered various fabric network latencies [2], [26], [35], [43]. Thus, we evaluated DeACT under the influence of various fabric latencies, Figure 15. An obvious observation is that when fabric network latency is less, the performance improvement with DeACT is also less and when the network latency is high, the performance improvement with DeACT is more. This is because when the fabric network latency is less, performance degradation in I-FAM itself is less, compared to E-FAM. This goes inversely when fabric latency is high. We see that even when fabric network latency is less, 100ns, DeACT achieves an improvement of 1.79x wrt to I-FAM. In contrast, when the network latency is 6us, DeACT speeds up I-FAM by 3.3x for *pf* benchmark.

4) Number of Nodes: The interconnecting fabric connects multiple PEs to the decoupled FAM modules. A single FAM module is expected to be part of a single memory pool. FAM architectures are constructed with multiple such memory pools and PEs. The performance of FAM architectures depends on the number of PEs and the number of memory pools. We maintained memory pools directly proportional to the number of

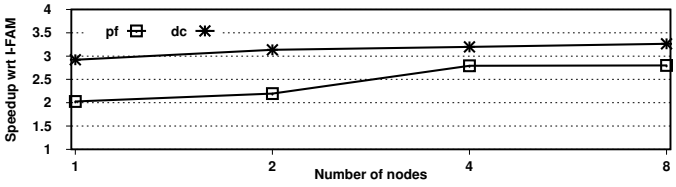


Fig. 16: Impact of increasing the number of nodes on performance.

nodes and each node has four PEs. For instance, an eight node system consists of 32 PEs and eight FAM modules. Each of the PEs accesses any of the memory pools. Memory pools and the PEs are connected through a common fabric network. The delay in accessing FAM depends on the number of nodes sharing the fabric interface and memory. Although scalability is beyond the scope of this paper we evaluated our approach when multiple nodes (up to 8) share the fabric. As the number of PEs sharing the fabric increase, we observe that the slowdown due to I-FAM is higher. This is due to more cycles are consumed to fetch FAM page table entries since, the fabric network and memory are shared by the nodes. As a result, the performance improvement with DeACT is more since, DeACT avoids accessing FAM for page table entries for most of the time. When the fabric network and memory are allocated to only one node the performance improvement with DeACT is 2.92x for *dc* benchmark and it increases to 3.26x when fabric network and memory are shared between 8 nodes.

VI. Discussion

We proposed large pages (1GB) in the global memory for only shared pages to reduce the memory occupied by the shared pages access control bitmap. Ideally, large pages would also reduce address translation overhead. But for FAM systems this is not completely true. Hence, we shed some light to discuss how large pages for shared memory is achievable and would not provide benefits for non-shared global memory. Also, we discuss how to invalidate system-level translations and ACM to migrate jobs smoothly between the nodes by introducing logical node numbers for the jobs in FAM systems. We even discuss the scalability and emerging fabric network aspects of our approach in FAM systems.

Shared Pages: FAM systems allow for easy sharing of data by registering to a common memory region in the global memory, shared pages. The proposed access control bitmap mechanism for shared pages is also applicable for baseline I-FAM, wherein the address translations are not decoupled from the access control, since the system-level access control alone is not sufficient to indicate how many nodes the page is shared with. System-level access control can only indicate if the page is shared or not. Also, because the modern applications share a lot of data frequently, large shared pages, as proposed, would lessen the burden of the memory manager to keep a track of the shared pages. The size of 1GB for shared pages is chosen as a trade-off between storage overhead for maintaining bitmasks of all sharers of a page and the number of pages that can be

shareable in the system. Too large would cause less sharing options and shareable pages, while too small size would cause high storage overheads.

Page Migration: A requirement for hybrid cloud systems is to migrate jobs between the nodes [24]. With I-FAM, it is easy to move jobs to another system since host/system data need not be moved. The only overhead is to invalidate node addresses of the job in both node (value) and fabric (tag) page tables. With the DeACT method, node-level translations are invalidated similar to I-FAM. However, system-level translations are invalidated by a) accessing fabric translation cache at DRAM b) invalidating access control at STU cache, and c) updating access control of migrating jobs with the target node at the global memory. Hence, the overhead of system-level mapping shootdown is accessing global memory separately to update access control of migrating jobs and excess DRAM writes to invalidate system-level mappings at in-memory FAM translation cache.

To allow smooth migration of jobs, logical node IDs per job can be used. Logical node IDs are assigned to jobs by the resource managers [17], [29]. For such a scheme migrating jobs requires only assigning new logical node ID in the destination node and removing assigned logical node ID in the old node, apart from invalidating page mappings.

Scalability: As asserted, although scalability is beyond the scope of this paper, we show performance improvements for 8 nodes with the DeACT approach. We show that as the number of nodes sharing Gen-Z fabric network increases, the improvement with DeACT also increases since most of the system-level address translations avoid fabric network traffic and contention at FAM pools generated by other nodes in our approach. As such, our hypothesis is valid even when more nodes share fabric interconnect.

Large Pages and Memory Regions: While using large pages improves system performance since TLBs caches page mapping for larger memory, it has some drawbacks [27], [48]. FAM systems have unique concerns in addition to the concerns with large pages in traditional in-node memory systems. To improve node proximity of data, pages are placed near the computing units, in local memory. However, achieving node proximity of data with large pages in FAM systems faces the following concerns: a) As local memory size is limited it can host only a fewer number of large pages which covers a lesser number of applications. b) Applications do not frequently use the entire large page, but access only specific portions of the large page frequently. This results in poor utilization of the DRAM which is critical for disaggregated memory systems and leads to wastage of network resources while moving pages from FAM to the DRAM. However, pages that are shared with other nodes have to be placed in FAM only, since there are accessed by various nodes. Hence, large pages for shared pages would not generate additional concerns unlike pages that are not shared.

VII. Related Work

Recently, disaggregating memory from PEs has been explored as an alternative memory architecture to overcome various operational and scalability challenges of in-node memory architectures [1], [2], [35], [38], [39], [43]. Works such as [14], [18], [20], [30] discuss and explore fast interconnect to enable decoupling memory. However, there has been limited work discussing virtual memory and security for such systems. Lim et al. [38] discussed two stage address translations for FAM systems, but their approach is limited to using remote memory merely as a swap space. Lim et al. [38] also proposed fine-grained remote memory accesses, which is similar to E-FAM and is not secure as discussed. Shan et al. [43] proposed decoupled OS for FAM systems and the address translations are performed by the FAM modules. However, for such a scheme to work the caches must be virtually indexed and virtually tagged which is not adopted and is not a practical design. Also, it requires significant changes to the OS. Aguilera et al. [2] invalidated virtual memory paging for such huge memory designs. Aguilera et al. also proposed fixed virtual address regions for the nodes [1]. However, this requires modifications to applications' binaries. In this paper, we discuss virtual memory support for FAM architectures with two stage address translations (I-FAM) and propose DeACT scheme to accelerate address translations.

Decoupling access control and address mappings have been explored previously. Alam et al. [4] discussed decoupled address control from address mapping allowing applications to perform address translations by itself. Olson et al. [41] proposed an approach to sandbox accelerators by providing them with flat address space. DeACT uniquely leverages the architecture layout of FAM architectures when decoupling access control from translation; it allows fast caching of translations in local nodes' main memories and maintains access control in the trusted area (i.e., at system-level). Additionally, DeACT supports data sharing across nodes and leverages system-level translation units at the fabric.

Although virtual machine guests are different from nodes in FAM systems, in both cases virtual addresses are translated at two stages to access memory. A significant amount of work has been done to improve the performance of virtualized conventional machines. Bhargava et al. [11] accelerate 2D PTW by studying reuse of page entry references and extend PTW caches to temporarily cache nested dimension. Ahn et al. [3] revisited hardware-assisted page walks by speculative shadow paging mechanism, called speculative inverted shadow paging, which is backed by non-speculative flat nested page tables. The speculative mechanism provides a direct translation with a single memory reference for common cases and eliminates the page table synchronization overheads. Agile paging is proposed by Gandhi et al. [25]. Agile paging allows a virtualized page walk to start with the shadow paging for stable upper levels of the page table and allows switching in the same page walk to nested paging for lower levels of the page table which receive frequent updates. This way agile paging makes use of both shadow paging and nested paging. While these approaches improve the system performance, these are proposed for virtual machines and our approach is orthogonal to these schemes.

VIII. Conclusion

Disaggregated memory systems is a promising future architecture that provides a path for future memory systems by solving current memory concerns in scalability, with the sharing of huge data sets such as social graphs, as well as complex scientific data-sets in HPC. The challenges associated with their design in an HPC context is how to best utilize the resources of the system and balance these against the ever-increasing demands to achieve better performance. To this end, we discuss virtual memory management in disaggregated memory systems and propose solutions to speed up address translations and provide security for such systems. While approaches like [3], [25] reduce the number of memory accesses to fetch address mapping in virtualized systems, they target native virtualized systems. Due to the hierarchical nature of the memory, disaggregated memory systems have their own challenges supporting virtual memory. We show that exposing global memory to the nodes needs OS alterations and compromise security from neighbor nodes. The virtual memory approach for disaggregated memory (indirect memory access) does not ask for OS modifications and provides security from neighbor nodes, but performs poorly. Although virtual memory support is discussed for disaggregated memory systems in [1], [2], [38], [43], in such approaches remote memory is merely used as swap space and required application and OS changes. We proposed a decoupled address translation and ACM approach to improving the performance of I-FAM. We show that an improved spatial locality of system-level translations by decoupling the system-level address translations from system-level ACM and caching the decoupled FAM translations in the local memory. We also explore ACM caching in STU cache to improve the performance. Overall, we achieved a performance improvement of up to 4.59x (1.8x on average).

IX. Acknowledgments

This work has been funded through Sandia National Laboratories (Contract Number 1844457). Sandia National Laboratories is a multi-mission laboratory managed and operated by National Technology and Engineering Solutions of Sandia, LLC., a wholly owned subsidiary of Honeywell International,Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525. Part of the work was funded by NSF's CNS-1814417 and CNS-1908471, and Office of Naval Research (ONR). This paper describes objective technical results and analysis. Any subjective views or opinions that might be expressed in the paper do not necessarily represent the views of the U.S. Department of Energy or the United States Government.

References

[1] M. K. Aguilera, N. Amit, I. Calciu, X. Deguillard, J. Gandhi, S. Novakovic, A. Ramanathan, P. Subrahmanyam, L. Suresh, K. Tati *et al.*, "Remote regions: a simple abstraction for remote memory," in *2018 {USENIX} Annual Technical Conference ({USENIX} {ATC} 18)*, 2018, pp. 775–787.

[2] M. K. Aguilera, N. Amit, I. Calciu, X. Deguillard, J. Gandhi, P. Subrahmanyam, L. Suresh, K. Tati, R. Venkatasubramanian, and M. Wei, "Remote memory in the age of fast networks," in *Proceedings of the 2017 Symposium on Cloud Computing*, 2017, pp. 121–127.

[3] J. Ahn, S. Jin, and J. Huh, "Revisiting hardware-assisted page walks for virtualized systems," in *2012 39th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2012, pp. 476–487.

[4] H. Alam, T. Zhang, M. Erez, and Y. Etsion, "Do-it-yourself virtual memory translation," in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2017, pp. 457–468.

[5] A. Awad, A. Basu, S. Blagodurov, Y. Solihin, and G. H. Loh, "Avoiding tlb shootdowns through self-invalidating tlb entries," in *2017 26th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 2017, pp. 273–287.

[6] A. Awad, S. Blagodurov, and Y. Solihin, "Write-aware management of nvm-based memory extensions," in *Proceedings of the 2016 International Conference on Supercomputing*, 2016, pp. 1–12.

[7] A. Awad, S. Hammond, C. Hughes, A. Rodrigues, S. Hemmert, and R. Hoekstra, "Performance analysis for using non-volatile memory dimms: opportunities and challenges," in *Proceedings of the International Symposium on Memory Systems*, 2017, pp. 411–420.

[8] A. Awad, G. R. Voskuilen, S. D. Hammond, and R. J. Hoekstra, "Samba: A detailed memory management unit (mmu) for the sst simulation framework." Sandia National Lab.(SNL-NM), Albuquerque, NM (United States), Tech. Rep. SAND2017-0002, 2017.

[9] D. H. Bailey, "Nas parallel benchmarks," in *Encyclopedia of Parallel Computing*. Springer, 2011, pp. 1254–1259.

[10] S. Beamer, K. Asanović, and D. Patterson, "The gap benchmark suite," *arXiv preprint arXiv:1508.03619*, 2015.

[11] R. Bhargava, B. Serebrin, F. Spadini, and S. Manne, "Accelerating two-dimensional page walks for virtualized systems," in *ACM SIGOPS Operating Systems Review*, vol. 42, no. 2. ACM, 2008, pp. 26–35.

[12] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The parsec benchmark suite: Characterization and architectural implications," in *Proceedings of the 17th international conference on Parallel architectures and compilation techniques*. ACM, 2008, pp. 72–81.

[13] C. Bienia and K. Li, "Parsec 2.0: A new benchmark suite for chip-multiprocessors," in *Proceedings of the 5th Annual Workshop on Modeling, Benchmarking and Simulation*, vol. 2011, 2009.

[14] M. S. Birrittella, M. Debbage, R. Huggahalli, J. Kunz, T. Lovett, T. Rimmer, K. D. Underwood, and R. C. Zak, "Intel® omni-path architecture: Enabling scalable, high performance fabrics," in *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*. IEEE, 2015, pp. 1–9.

[15] S. Chen, L. Liu, W. Zhang, and L. Peng, "Architectural support for nvram persistence in gpus," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 5, pp. 1107–1120, 2019.

[16] D. Comperchio and J. Stevens, "Emerging computing technologies: Hewlett-packard's "the machine" project," in *HP Discover 2014 conference held in Las Vegas June 10-12*. Willdan Energy Solutions, 2014, pp. 1–4.

[17] A. Computing and G. Computing, "Torque resource manager," *online] http://www. adaptivecomputing. com*, 2015.

[18] C. Consortium *et al.*, "Cache coherent interconnect for accelerators (ccix)," *Online]. http://www. ccixconsortium. com*, 2017.

[19] G.-Z. Consortium *et al.*, "Gen-z overview," Tech. Rep., 2016.[Online]. Available: http://genzconsortium. org/wp-content ..., Tech. Rep., 2016.

[20] G.-Z. Consortium *et al.*, "Gen-z dram and persistent memory theory of operation," 2017.

[21] G. Consortium, "Gen-z zmmu and memory interleave," *Online]. https://genzconsortium.org/wp-content/uploads/2018/05/Gen-Z-MMU-and-Memory-Interleave.pdf*, July 2017.

[22] O. Consortium *et al.*, "Opencapi specifications," 2018.

[23] P. Express, "Address translation services revision 1.1," *Online]. https://composter.com.ua/documents/ats*, $1.1_26Jan09.pdf$, $January2009$.

[24] I. Foster, Y. Zhao, I. Raicu, and S. Lu, "Cloud computing and grid computing 360-degree compared," *arXiv preprint arXiv:0901.0131*, 2008.

[25] J. Gandhi, M. D. Hill, and M. M. Swift, "Agile paging: exceeding the best of nested and shadow paging," in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2016, pp. 707–718.

[26] P. X. Gao, A. Narayan, S. Karandikar, J. Carreira, S. Han, R. Agarwal, S. Ratnasamy, and S. Shenker, "Network requirements for resource

disaggregation," in *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, 2016, pp. 249–264.

[27] F. Gaud, B. Lepers, J. Decouchant, J. Funston, A. Fedorova, and V. Quéma, "Large pages may be harmful on {NUMA} systems," in *2014 {USENIX} Annual Technical Conference ({USENIX} {ATC} 14)*, 2014, pp. 231–242.

[28] B. Gras, K. Razavi, H. Bos, and C. Giuffrida, "Translation leak-aside buffer: Defeating cache side-channel protections with {TLB} attacks," in *27th {USENIX} Security Symposium ({USENIX} Security 18)*, 2018, pp. 955–972.

[29] H. Greenberg, M. Lang, L. Ionkov, and S. Blanchard, "Redfish—resilient dynamic distributed scalable system services for exascale."

[30] J. Gu, Y. Lee, Y. Zhang, M. Chowdhury, and K. G. Shin, "Efficient memory disaggregation with infiniswap," in *14th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 17)*, 2017, pp. 649–667.

[31] J. L. Henning, "SPEC CPU2006 benchmark descriptions," *ACM SIGARCH Computer Architecture News*, vol. 34, no. 4, pp. 1–17, sep 2006. [Online]. Available: https://doi.org/10.1145/1186736.1186737

[32] M. Heroux and R. Barrett, "Mantevo project," 2016.

[33] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher *et al.*, "Spectre attacks: Exploiting speculative execution," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 1–19.

[34] V. Kommareddy, C. Hughes, S. D. Hammond, and A. Awad, "Opal: A centralized memory manager for investigating disaggregated memory systems." Sandia National Lab.(SNL-NM), Albuquerque, NM (United States), Tech. Rep. SAND2018-9199, 2018.

[35] V. R. Kommareddy, S. D. Hammond, C. Hughes, A. Samih, and A. Awad, "Page migration support for disaggregated non-volatile memories," in *Proceedings of the International Symposium on Memory Systems*. ACM, 2019, pp. 417–427.

[36] V. R. Kommareddy, C. Hughes, S. Hammond, and A. Awad, "Investigating fairness in disaggregated non-volatile memories," in *2019 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2019, pp. 104–110.

[37] J. Kyathsandra and E. Dahlen, "Intel rack scale architecture overview," *Proc. INTEROP, Las Vegas, NV, 2–6 May 2013*, 2013.

[38] K. Lim, J. Chang, T. Mudge, P. Ranganathan, S. K. Reinhardt, and T. F. Wenisch, "Disaggregated memory for expansion and sharing in blade servers," in *ACM SIGARCH Computer Architecture News*, vol. 37, no. 3. ACM, 2009, pp. 267–278.

[47] M. Sutton, T. Ben-Nun, and A. Barak, "Optimizing parallel graph connectivity computation via subgraph sampling," in *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2018, pp. 12–21.

[39] K. Lim, Y. Turner, J. R. Santos, A. AuYoung, J. Chang, P. Ranganathan, and T. F. Wenisch, "System-level implications of disaggregated memory," in *High Performance Computer Architecture (HPCA), 2012 IEEE 18th International Symposium on*. IEEE, 2012, pp. 1–12.

[40] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown," *arXiv preprint arXiv:1801.01207*, 2018.

[41] L. E. Olson, J. Power, M. D. Hill, and D. A. Wood, "Border control: Sandboxing accelerators," in *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2015, pp. 470–481.

[42] A. F. Rodrigues, K. S. Hemmert, B. W. Barrett, C. Kersey, R. Oldfield, M. Weston, R. Risen, J. Cook, P. Rosenfeld, E. CooperBalls *et al.*, "The structural simulation toolkit," *ACM SIGMETRICS Performance Evaluation Review*, vol. 38, no. 4, pp. 37–42, 2011.

[43] Y. Shan, Y. Huang, Y. Chen, and Y. Zhang, "Legoos: A disseminated, distributed {OS} for hardware resource disaggregation," in *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*, 2018, pp. 69–87.

[44] D. Sharma, "Compute express link," *CXL Consortium White Paper.[Online]. Available: https://docs. wixstatic. com/ugd/0c1418 d9878707bbb7427786b70c3c91d5fbd1. pdf*, 2019.

[45] Y. Shiloach and U. Vishkin, "An o (log n) parallel connectivity algorithm," Computer Science Department, Technion, Tech. Rep., 1980.

[46] V. Shrivastav, A. Valadarsky, H. Ballani, P. Costa, K. S. Lee, H. Wang, R. Agarwal, and H. Weatherspoon, "Shoal: A network architecture for disaggregated racks," in *16th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 19)*, 2019, pp. 255–270.

[48] M. Talluri, S. Kong, M. D. Hill, and D. A. Patterson, "Tradeoffs in supporting two page sizes," in *[1992] Proceedings the 19th Annual International Symposium on Computer Architecture*. IEEE, 1992, pp. 415–424.

[49] J. Taylor, "Facebook's data center infrastructure: Open compute, disaggregated rack, and beyond," in *Optical Fiber Communication Conference*. Optical Society of America, 2015, p. W1D.5. [Online]. Available: http://www.osapublishing.org/abstract.cfm?URI=OFC-2015-W1D.5

[50] S. Xi, J. Wilson, C. Lu, and C. Gill, "Rt-xen: Towards real-time hypervisor scheduling in xen," in *Proceedings of the ninth ACM international conference on Embedded software*. ACM, 2011, pp. 39–48.